# AI ASSISTANT CODING - ASSIGNMENT-15.4

**Name:** Valaboju Harshavardhanu

**Roll No:** 2303A52354 **Batch:** 41

## Task 1: Initial API Setup for a Service Platform

### Prompt:

Create a basic Flask application. Implement a root endpoint /status that returns a JSON response indicating 'API is live'. Provide the necessary code to run this server locally within the Colab environment and include instructions on how to verify the /status endpoint's response using either a browser or an API testing tool.

### Code:

```python
from flask import Flask, jsonify

app = Flask(__name__)

@app.route('/status')
def status():
    return jsonify({"status": "API is live"})

if __name__ == '__main__':
    # In a Colab environment, '0.0.0.0' makes the server accessible within Colab's network.
    # 'debug=True' provides helpful debugging information.
    app.run(host='0.0.0.0', port=5000, debug=True)
```

```python
import requests
import time

# Give the server a moment to start
time.sleep(3)

try:
  response = requests.get('http://127.0.0.1:5000/status')
  print(response.json())
except requests.exceptions.ConnectionError:
  print("Could not connect to the Flask server. Make sure the Flask app cell is running.")
```

### Output:

```
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

### Justification:

This task ensures the backend environment is correctly configured and running. A simple /status endpoint acts as a **health check**, confirming that the server is active and ready to handle requests.

## Task 2: Create API to Add and View Resources

### Prompt:

Expand the existing Flask application to include a GET endpoint at /requests that retrieves all service requests, and a POST endpoint at /requests that adds a new service request. Each request object should have a title and priority. Provide the Flask application code in new cells, along with verification steps to demonstrate an empty list on initial GET, adding a request via POST, and retrieving the added request via a subsequent GET.

### Code:

Gemini

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

## Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

## Justification:

This establishes basic **CRUD (Create & Read)** functionality. It allows the system to store and retrieve service requests, which is essential for managing user issues in any service platform.

# Task 3: Modify Existing Resource Using AI Assistance

## Prompt:

Add PUT endpoint to update a request by index with proper error handling in Flask.

## Code:

```
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)

@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

## Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

### Justification:

Updating resources introduces **dynamic data handling**, enabling changes like priority updates. Proper error handling (e.g., invalid index) ensures the API is **robust and reliable**.

## Task 4: Delete Resource

### Prompt:

Add DELETE endpoint to remove a request by index with safe error handling.

### Code:

```python
from flask import Flask, jsonify, request

app = Flask(__name__)

# In-memory storage for service requests
requests = []
request_id_counter = 1

@app.route('/status')
def status():
    return jsonify({'status': 'ok', 'message': 'API is running'})

@app.route('/requests', methods=['POST'])
def create_request():
    global request_id_counter
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority'}), 400

    new_request = {
        'id': request_id_counter,
        'title': data['title'],
        'priority': data['priority']
    }
    requests.append(new_request)
    request_id_counter += 1
    return jsonify(new_request), 201

@app.route('/requests', methods=['GET'])
def get_requests():
    return jsonify(requests)
```

```python
@app.route('/requests/<int:request_id>', methods=['PUT'])
def update_request(request_id):
    data = request.get_json()
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    global requests
    original_length = len(requests)
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

### Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab cells might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

### Justification:

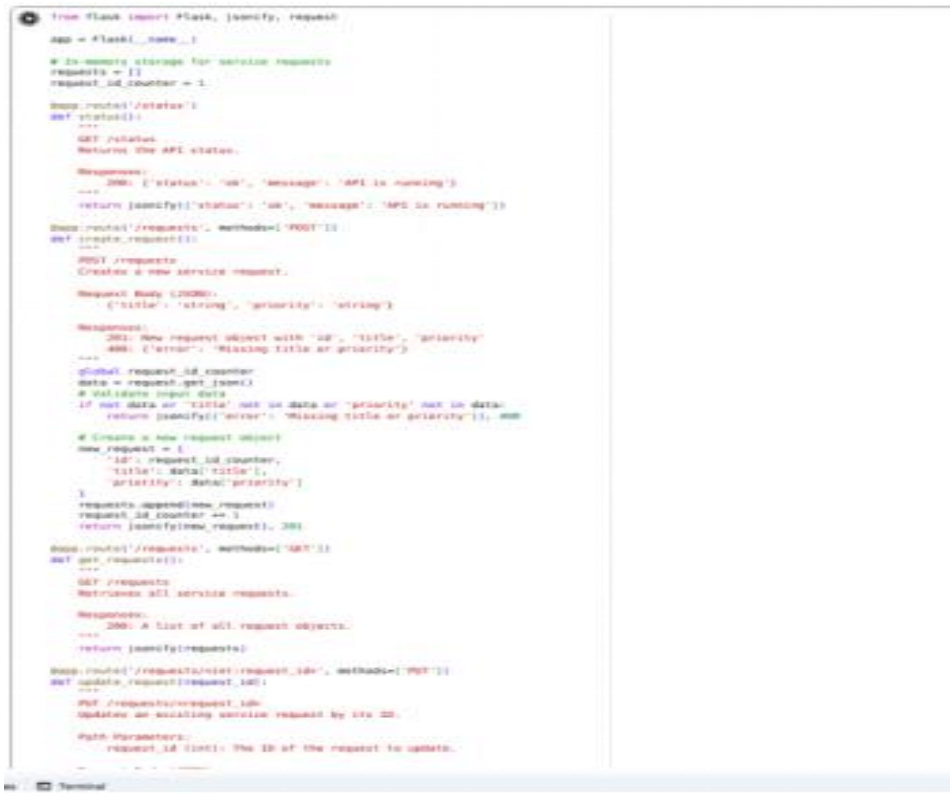
Deleting resolved requests helps maintain **clean and relevant data**. Safe deletion with error handling prevents crashes and ensures **data integrity**.

## Task 5: AI-Assisted API Documentation and Readability

### Prompt:

Add docstrings and comments to Flask API endpoints and optionally integrate Swagger documentation.

### Code:



```python
    # Validate input data
    if not data or 'title' not in data or 'priority' not in data:
        return jsonify({'error': 'Missing title or priority in request body'}), 400

    # Find and update the request
    for req in requests:
        if req['id'] == request_id:
            req['title'] = data['title']
            req['priority'] = data['priority']
            return jsonify(req), 200
    return jsonify({'error': 'Request not found'}), 404

@app.route('/requests/<int:request_id>', methods=['DELETE'])
def delete_request(request_id):
    """
    DELETE /requests/<request_id>
    Deletes a service request by its ID.

    Path Parameters:
        request_id (int): The ID of the request to delete.

    Responses:
        200: {'message': 'Request with id {request_id} deleted successfully'}
        404: {'error': 'Request not found'}
    """
    global requests
    original_length = len(requests)
    # Filter out the request to be deleted
    requests = [req for req in requests if req['id'] != request_id]
    if len(requests) < original_length:
        return jsonify({'message': f'Request with id {request_id} deleted successfully'}), 200
    return jsonify({'error': 'Request not found'}), 404

if __name__ == '__main__':
    # In Colab, you might need to use a different host to make it accessible
    # from the Colab environment. '0.0.0.0' is generally used for external access.
    # However, for local testing within Colab, '127.0.0.1' or even not specifying
    # the host might work depending on the Colab environment setup.
    # For a simple run, you can just use app.run(debug=True).
    # If you intend to expose it via ngrok or similar, you'd configure that separately.
    print("To run the app, execute 'python your_file_name.py' in a terminal if this were a script.")
    print("In Colab, you can run it directly, but accessing it from outside Colab calls might require port forwarding or ngrok.")
    app.run(host='0.0.0.0', port=5000, debug=True)
```

## Output:

```
To run the app, execute 'python your_file_name.py' in a terminal if this were a script.
In Colab, you can run it directly, but accessing it from outside Colab calls might require port forwarding or ngrok.
 * Serving Flask app '__main__'
 * Debug mode: on
2025-07-07 07:07:07,[illegible] WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:5000
 * Running on http://172.28.0.12:5000
2025-07-07 07:07:07,[illegible] Press CTRL+C to quit
2025-07-07 07:07:07,[illegible] * Restarting with watchdog (inotify)
```

## Justification:

Good documentation improves **developer experience and collaboration**. It makes the API easy to understand, use, and maintain, while tools like Swagger provide **interactive testing and clarity**.